

Dimensionality Reduction using Principal Component Analysis (PCA)

Firuz Kamalov

MAI 600: Mathematics and Statistics for AI

November 27, 2025



Lecture Outline

Motivation: Why Dimensionality Reduction?

PCA: Intuition

Mathematical Formulation

PCA Algorithm and Examples

Kernel PCA

Practical Considerations and Limitations

Summary

The Curse of Dimensionality

Modern AI systems often work with very high-dimensional data:

- ▶ Images: $28 \times 28 = 784$ features, or even $224 \times 224 \times 3$.
- ▶ Text: Hundreds or thousands of word features.
- ▶ Sensor data: Many correlated signals.

Problems with high dimensions:

- ▶ Hard to visualize and understand.
- ▶ Models become complex and prone to overfitting.
- ▶ Training time and memory usage increase.

Goal: **Compress** data into fewer dimensions while retaining most of the information.

Dimensionality Reduction in AI Pipelines

Typical workflow in AI:

Raw Data → **Preprocessing** → **Dimensionality Reduction** → **Model**

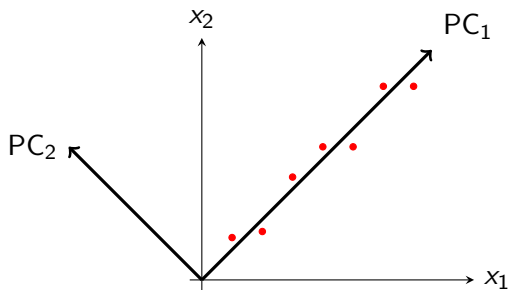
Examples:

- ▶ Compress image features before feeding to a classifier.
- ▶ Reduce noise in sensor data by keeping only main directions of variation.
- ▶ Visualize high-dimensional embeddings (e.g., word vectors) in 2D.

PCA is one of the most classical and widely used methods for this purpose.

A Simple 2D Example

Suppose we have 2D data that roughly lies along a diagonal line.



Idea:

- ▶ The data vary **most** along the diagonal direction.
- ▶ Each point can be described by its coordinate along this direction.
- ▶ This effectively reduces 2D data to a 1D representation.

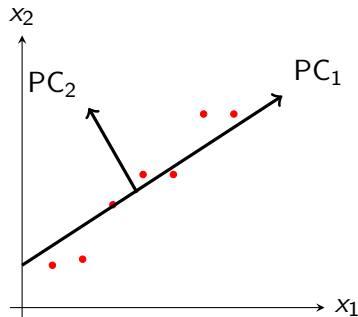
Principal Component Analysis finds:

- ▶ New axes (directions in space),
- ▶ That are orthogonal to each other,
- ▶ Ordered by how much variance of the data they explain.

Key idea:

- ▶ First principal component (PC1): direction of **maximum variance**.
- ▶ Second principal component (PC2): direction of maximum remaining variance, orthogonal to PC1.
- ▶ And so on.

Geometric View



Each point can be projected onto the new axes.

- ▶ Keeping only PC_1 gives a 1D representation capturing the main structure.
- ▶ Keeping PC_1 and PC_2 gives a rotated 2D coordinate system.

Data Matrix and Centering

Let

$$X = \begin{bmatrix} - & \mathbf{x}_1^\top & - \\ - & \mathbf{x}_2^\top & - \\ & \vdots & \\ - & \mathbf{x}_n^\top & - \end{bmatrix} \in \mathbb{R}^{n \times d}$$

be the data matrix.

- ▶ n = number of observations (samples).
- ▶ d = number of features (dimensions).

We first **center** the data:

$$\bar{\mathbf{x}} = \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i, \quad X \leftarrow X - \mathbf{1}_n \bar{\mathbf{x}}^\top.$$

From now on we assume that X is centered. If not, we subtract the mean as above.

Optimization View of the First Principal Component

Assume $X \in \mathbb{R}^{n \times d}$ is centered.

The first principal component direction $\mathbf{w}_{(1)}$ is defined as

$$\mathbf{w}_{(1)} = \arg \max_{\|\mathbf{w}\|=1} \{ \|X\mathbf{w}\|^2 \}.$$

Interpretation:

- ▶ $X\mathbf{w}$ are the *projections* of the data onto direction $\mathbf{w}$.
- ▶ $\|X\mathbf{w}\|^2$ is proportional to the variance of these projections.
- ▶ Thus, $\mathbf{w}_{(1)}$ is the unit vector along which the data have **maximum variance**.

Solution via Eigenvalues and Eigenvectors

A standard result in linear algebra:

- ▶ $\mathbf{w}_{(1)}$ is the **largest eigenvector** of $X^\top X$, where $X^\top X$ is the covariance matrix of X .

Next components:

- ▶ $\mathbf{w}_2$ is the eigenvector corresponding to λ_2 , orthogonal to $\mathbf{w}_1$.
- ▶ Continue until λ_d and $\mathbf{w}_d$.

The eigenvalues $\lambda_1, \dots, \lambda_d$ measure how much variance each principal component explains.

The PCA Algorithm

Given data matrix $X \in \mathbb{R}^{n \times d}$:

1. **Center**: form $X \leftarrow X - \mathbf{1}_n \bar{\mathbf{x}}^\top$.
2. **Compute** covariance matrix $\text{Cov} = X^\top X$.
3. **Find eigenvalues and eigenvectors** of Cov .
4. **Sort** eigenvalues in decreasing order $\lambda_1 \geq \dots \geq \lambda_d$.
5. **Form** k -dimensional subspace using eigenvectors $\mathbf{w}_1, \dots, \mathbf{w}_k$.
6. **Project** data: $Z = XW_k$, where $W_k = [\mathbf{w}_1 \ \dots \ \mathbf{w}_k]$.

Numeric Example: 2D to 1D

Consider $n = 4$ points in $\mathbb{R}^2$:

$$X = \begin{bmatrix} 2 & 1 \\ 3 & 2 \\ 4 & 3 \\ 5 & 4 \end{bmatrix}.$$

- Compute the mean:

$$\bar{\mathbf{x}} = \left(\frac{2 + 3 + 4 + 5}{4}, \frac{1 + 2 + 3 + 4}{4} \right) = (3.5, 2.5).$$

- Center the data: $X = X - \mathbf{1}_4 \bar{\mathbf{x}}^\top$.
- Compute $\text{Cov} = X^\top X$.

The largest eigenvector of Cov is close to $(1, 1)^\top$, capturing the diagonal trend.

Explained Variance

Let $\lambda_1, \dots, \lambda_d$ be eigenvalues of S in decreasing order.

- ▶ Total variance:

$$\text{Total Var} = \sum_{j=1}^d \lambda_j.$$

- ▶ Variance explained by component j :

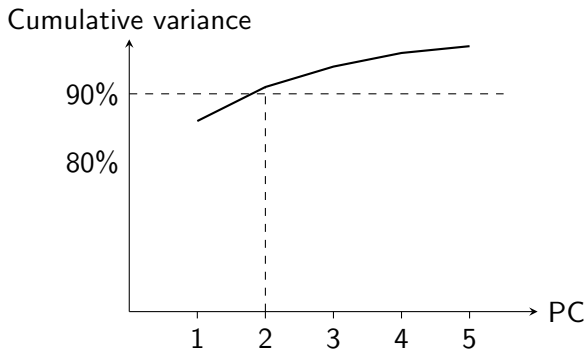
$$\text{Explained Var}_j = \frac{\lambda_j}{\sum_{i=1}^d \lambda_i}.$$

- ▶ Cumulative variance of first k components:

$$\text{Cumulative}_k = \frac{\sum_{j=1}^k \lambda_j}{\sum_{i=1}^d \lambda_i}.$$

In practice, we choose k such that Cumulative_k exceeds a threshold (for example, 90–95%).

Illustration: Choosing Number of Components



In this example, 2 components already explain about 90% of the variance.

Another way to compute PCA uses the **Singular Value Decomposition (SVD)**.

$$X = U\Sigma V^T,$$

where

- ▶ $U \in \mathbb{R}^{n \times n}$ has orthonormal columns,
- ▶ $\Sigma \in \mathbb{R}^{n \times d}$ is diagonal (non-negative singular values),
- ▶ $V \in \mathbb{R}^{d \times d}$ has orthonormal columns.

Then:

- ▶ Columns of V are the principal directions (eigenvectors of Cov).
- ▶ Singular values relate to eigenvalues: $\lambda_j = \frac{\sigma_j^2}{n-1}$.
- ▶ Projection onto first k PCs: $Z = XV_k$.

Example: Image Compression

Consider grayscale images of size 28×28 :

- ▶ Each image is a point in $\mathbb{R}^{784}$.
- ▶ Many pixels are highly correlated (neighboring pixels similar).

PCA approach:

- ▶ Flatten each image into a 784-dimensional vector.
- ▶ Stack into a matrix X and apply PCA.
- ▶ Keep only, say, 50 principal components: $k = 50$.

Result:

- ▶ We can store each image using 50 numbers instead of 784.
- ▶ We can approximately reconstruct the original image from these 50 components.

Feature Selection vs. PCA

Feature Selection

- ▶ Keeps a subset of the original features.
- ▶ Features remain directly interpretable.
- ▶ Can be supervised or unsupervised.
- ▶ Methods: correlation filters, mutual information, RFE, LASSO.
- ▶ Goal: identify the most relevant predictors.

PCA (Feature Extraction)

- ▶ Creates new features (principal components).
- ▶ Components are linear combinations of original variables.
- ▶ Unsupervised: maximizes variance, not prediction accuracy.
- ▶ Removes multicollinearity.
- ▶ Goal: reduce dimensionality with minimal information loss.

Key Idea: Feature selection keeps original features; PCA constructs new uncorrelated ones.

Motivation for Kernel PCA

- ▶ Standard PCA captures only *linear* structure.
- ▶ Many real datasets lie on *nonlinear manifolds*.
- ▶ Idea: map data into a high-dimensional feature space where linear PCA becomes effective.
- ▶ Use the **kernel trick** to avoid computing the high-dimensional mapping explicitly.

Nonlinear Mapping

- ▶ Let $\phi : \mathbb{R}^d \rightarrow \mathcal{H}$ be a nonlinear feature map.
- ▶ Apply PCA to the transformed data $\phi(X)$.
- ▶ Only inner products $\langle \phi(x_i), \phi(x_j) \rangle$ are required.
- ▶ They are computed using a kernel function:

$$k(x_i, x_j) = \langle \phi(x_i), \phi(x_j) \rangle.$$

- ▶ Polynomial kernel:

$$k(x, y) = (x^\top y + c)^p$$

- ▶ RBF (Gaussian) kernel:

$$k(x, y) = \exp(-\gamma \|x - y\|^2)$$

- ▶ Sigmoid kernel:

$$k(x, y) = \tanh(\alpha x^\top y + \beta)$$

- ▶ Kernel PCA operates entirely through the kernel matrix $K_{ij} = k(x_i, x_j)$.

Kernel PCA Algorithm

1. Compute kernel matrix:

$$K_{ij} = k(x_i, x_j).$$

2. Center the kernel matrix:

$$\tilde{K} = K - \mathbf{1}K - K\mathbf{1} + \mathbf{1}K\mathbf{1}.$$

3. Solve eigenvalue problem:

$$\tilde{K}v = \lambda v.$$

4. Compute principal component of a new point:

$$\text{PC}_k(x) = \sum_{i=1}^n v_{ik} k(x_i, x).$$

Kernel PCA Illustration

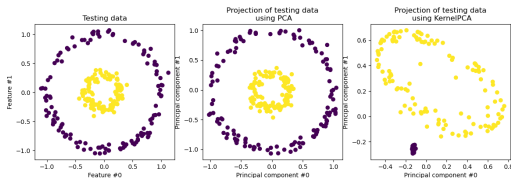


Figure: By using an RBF kernel, we expect that the projection will unfold the dataset.

Practical Tips for Using PCA

- ▶ **Standardization:** If features have very different scales, standardize before PCA.
- ▶ **Train/Test Splits:** Fit PCA on training data only, then apply the same transformation to test data.
- ▶ **Number of Components:** Use explained variance curves or cross-validation.
- ▶ **Interpretation:** Principal components are linear combinations of original features; sometimes hard to interpret.

PCA is widely available in libraries (e.g., scikit-learn) and is easy to integrate into AI workflows.

Limitations of PCA

- ▶ **Linear method:** PCA captures only linear relationships.
- ▶ **Global variance:** It focuses on directions of maximum variance, not necessarily those most relevant for a prediction task.
- ▶ **Sensitivity to outliers:** A few extreme points can affect the principal components.
- ▶ **Unsupervised:** It does not use class labels; some variance may be irrelevant for classification.

Nonlinear methods (kernel PCA, t-SNE, UMAP) can capture more complex structures, but PCA remains an essential baseline.

Summary

- ▶ Dimensionality reduction is crucial for handling high-dimensional data in AI.
- ▶ PCA finds orthogonal directions of maximum variance in the data.
- ▶ Mathematically, PCA is based on eigenvalues and eigenvectors of the covariance matrix (or SVD).
- ▶ We can reduce dimensionality by projecting onto the first k principal components while retaining most of the variance.
- ▶ PCA is simple, efficient, and widely used, but it is linear and unsupervised.

Thank you. Questions?